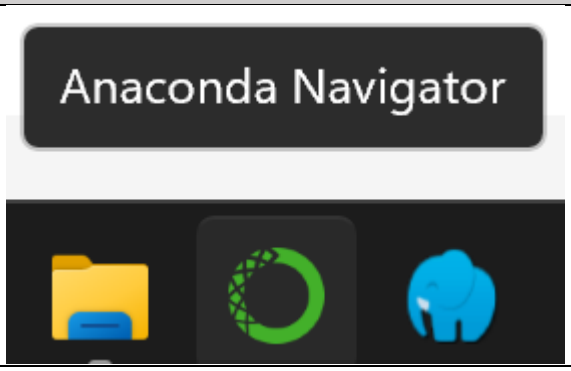
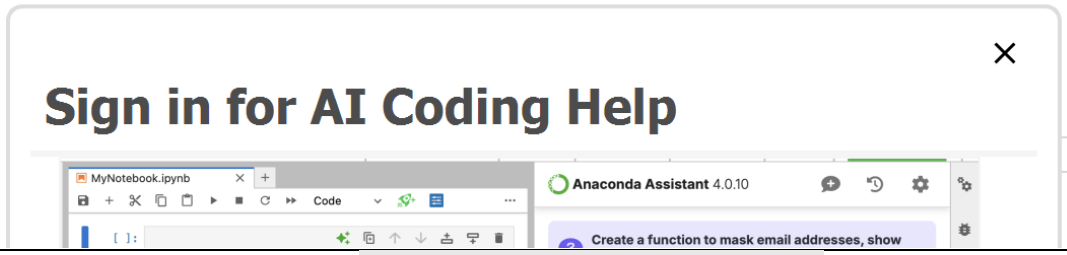
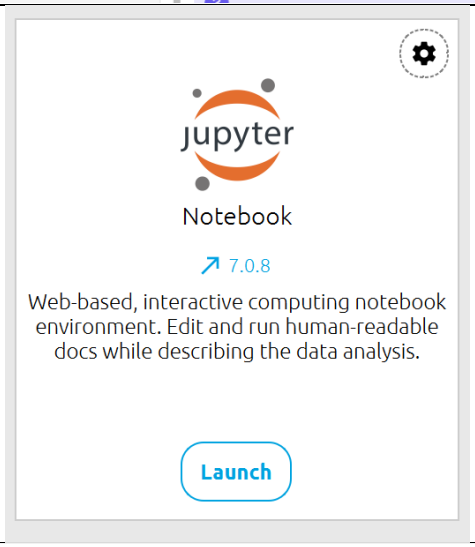
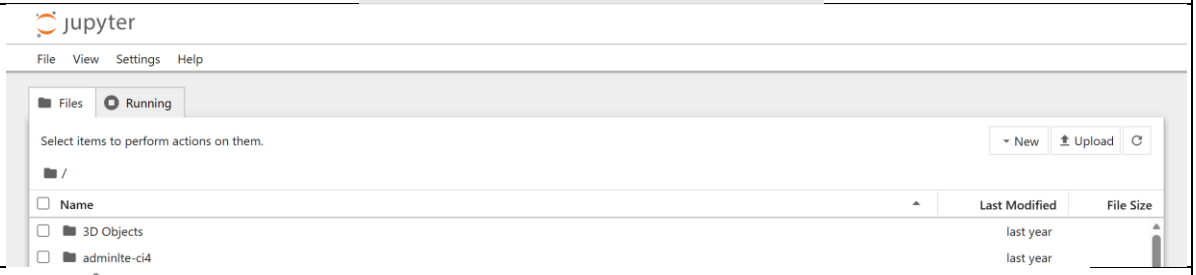
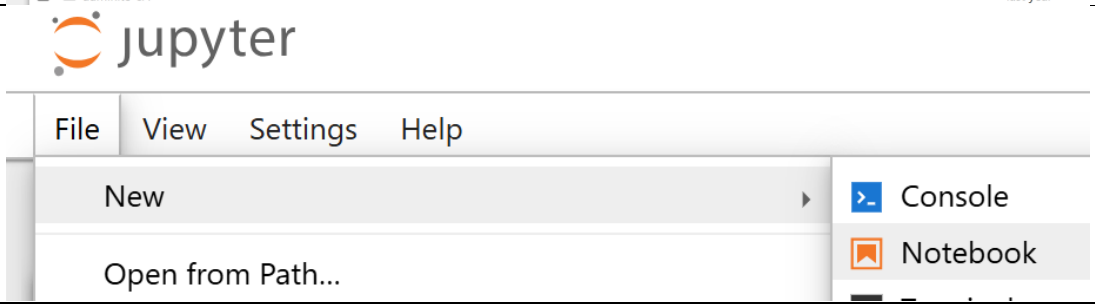
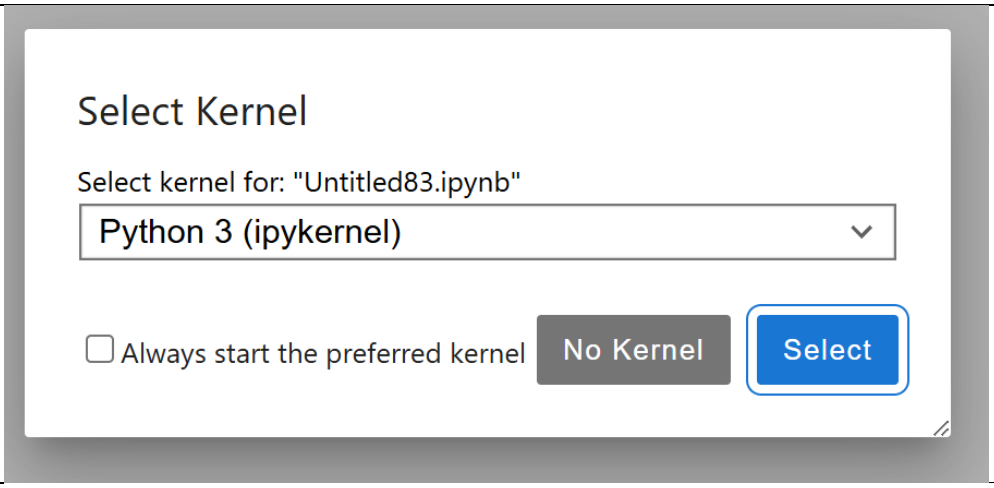
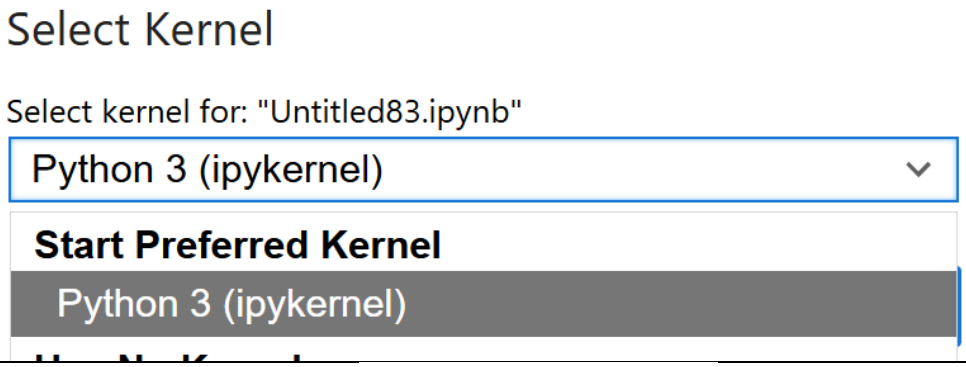
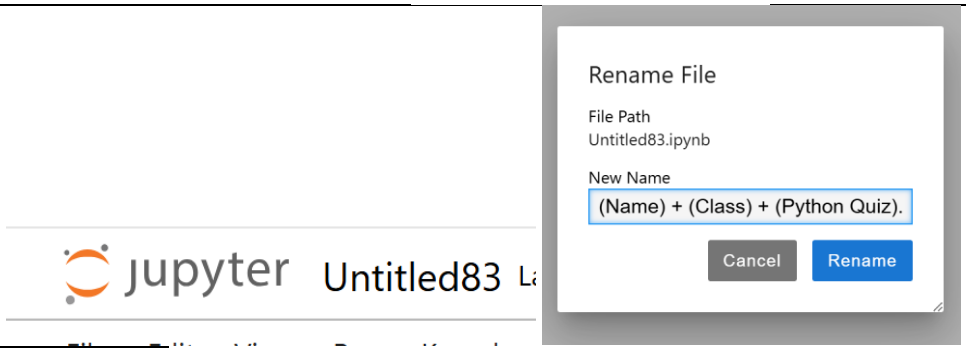
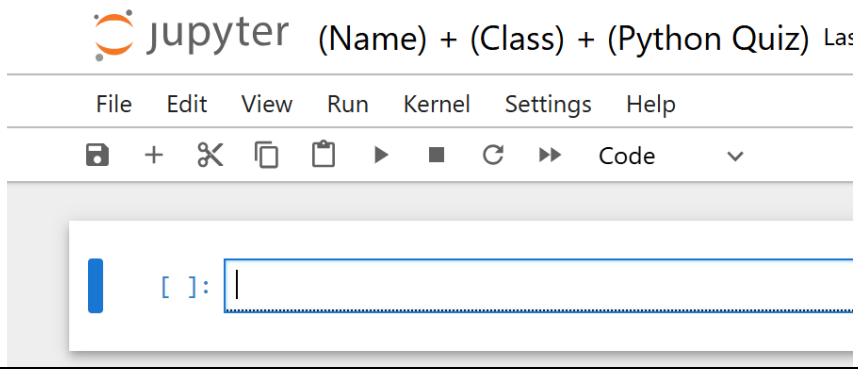
No.	Steps	Picture
1	Open Anaconda Navigator	
2	Close pop-up Sign in	
3	Scroll to find Jupyter Notebook and Click Launch	
4	Wait until you see the jupyter notebook	
5	File > New > Notebook	

6	Wait until you see pop-up kernel	
7	You can use any “PYTHON ” kernel	
8	Click Select	
9	Change the “Untitled” and rename	
10	Go to the cell code	

Bagian 1: Mengeksplorasi Parameter Statistik dari Data

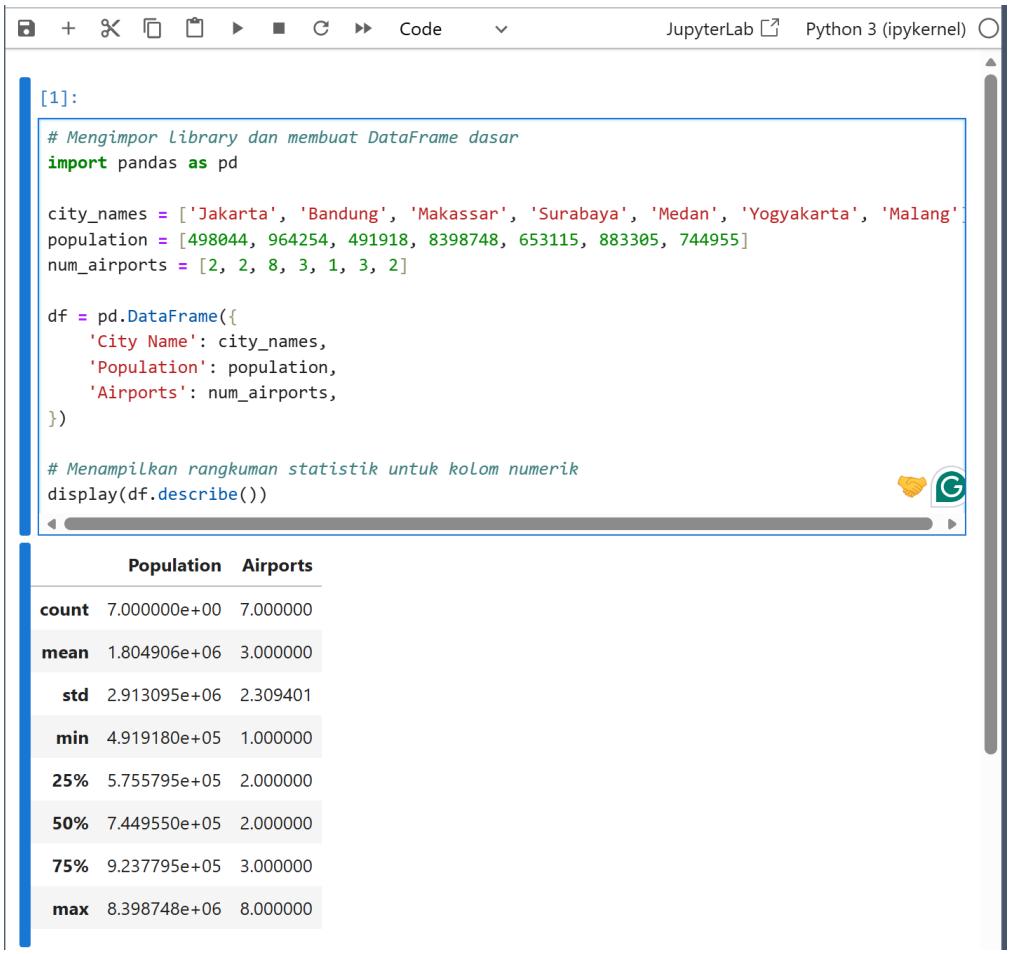
Cell 1: Membuat Data & Rangkuman Statistik

```
# Mengimpor library dan membuat DataFrame dasar
import pandas as pd

city_names = ['Jakarta', 'Bandung', 'Makassar', 'Surabaya', 'Medan', 'Yogyakarta', 'Malang']
population = [498044, 964254, 491918, 8398748, 653115, 883305, 744955]
num_airports = [2, 2, 8, 3, 1, 3, 2]

df = pd.DataFrame({
    'City Name': city_names,
    'Population': population,
    'Airports': num_airports,
})

# Menampilkan rangkuman statistik untuk kolom numerik
display(df.describe())
```



	Population	Airports
count	7.000000e+00	7.000000
mean	1.804906e+06	3.000000
std	2.913095e+06	2.309401
min	4.919180e+05	1.000000
25%	5.755795e+05	2.000000
50%	7.449550e+05	2.000000
75%	9.237795e+05	3.000000
max	8.398748e+06	8.000000

Cell 2: Rangkuman Statistik Menyeluruh

 # Menampilkan rangkuman statistik dengan menyertakan kolom non-numerik (kategorik)

display(df.describe(include="all"))

Code JupyterLab Python 3 (ipykernel)

[3]:

```
# Menampilkan rangkuman statistik dengan menyertakan kolom non-numerik (kategorik)
display(df.describe(include="all"))
```

	City Name	Population	Airports
count	7	7.000000e+00	7.000000
unique	7	NaN	NaN
top	Jakarta	NaN	NaN
freq	1	NaN	NaN
mean	NaN	1.804906e+06	3.000000
std	NaN	2.913095e+06	2.309401
min	NaN	4.919180e+05	1.000000
25%	NaN	5.755795e+05	2.000000
50%	NaN	7.449550e+05	2.000000
75%	NaN	9.237795e+05	3.000000
max	NaN	8.398748e+06	8.000000

Cell 3: Visualisasi Histogram

 # Membuat plot histogram dari kolom numerik pada DataFrame untuk memeriksa distribusi

df.hist()

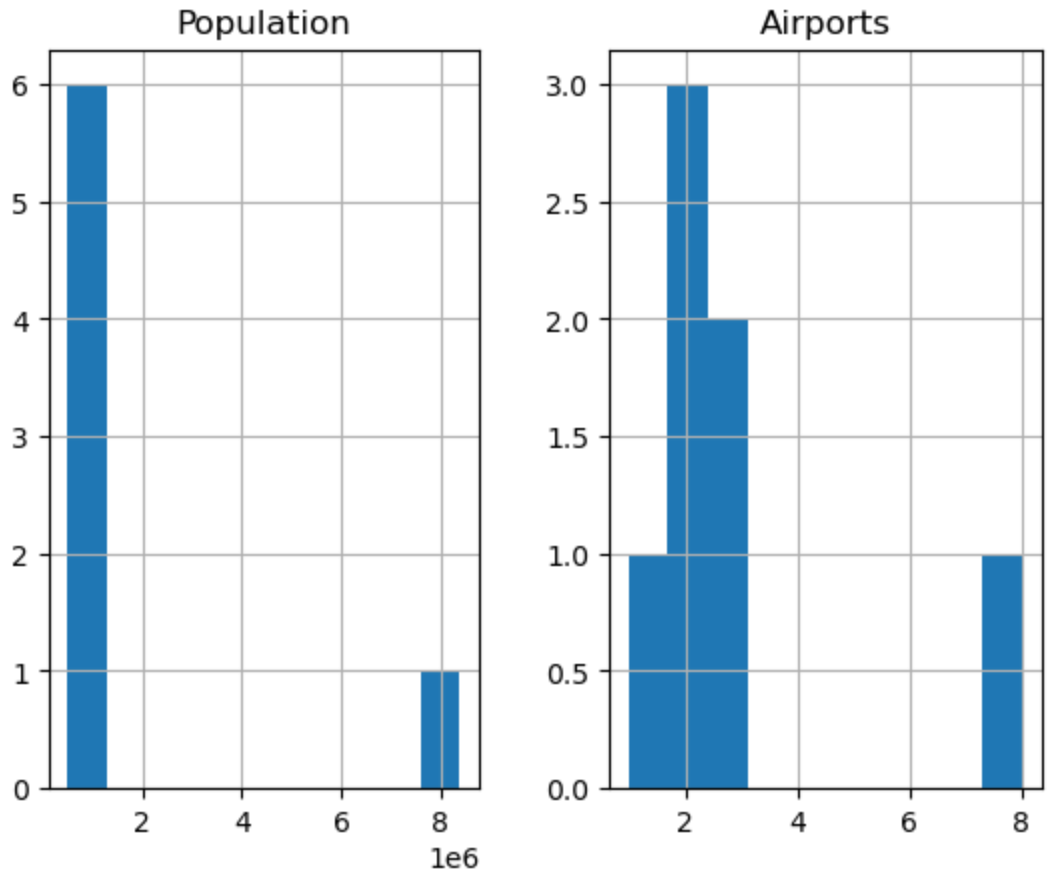
Code JupyterLab Python 3 (ipykernel)

[5]:

```
# Membuat plot histogram dari kolom numerik pada DataFrame untuk memeriksa distribusi
df.hist()
```

[5]:

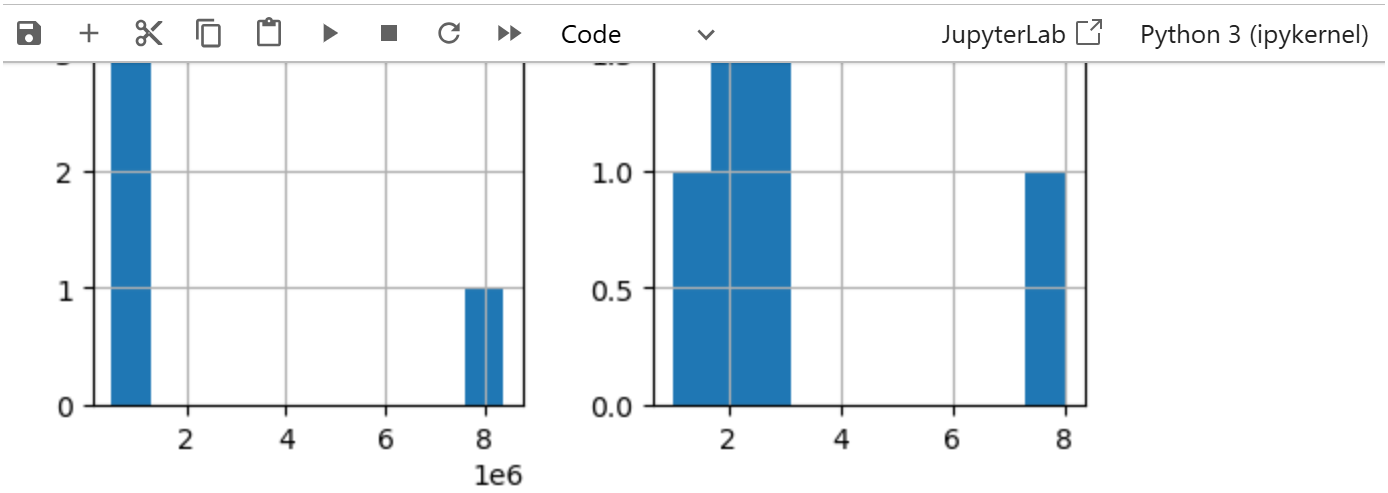
```
array([[<Axes: title={'center': 'Population'}>,
        <Axes: title={'center': 'Airports'}>]], dtype=object)
```



Cell 4: Memeriksa Korelasi

 # Memeriksa korelasi antar data numerik (menggunakan numeric_only=True agar tidak error di Pandas versi terbaru)

display(df.corr(numeric_only=True))



[7]:

```
# Memeriksa korelasi antar data numerik (menggunakan numeric_only=True agar tidak error di P
display(df.corr(numeric_only=True))
```

	Population	Airports
Population	1.000000	-0.026109
Airports	-0.026109	1.000000

Bagian 2: Mengelompokkan Data (Pivot Table)

Cell 5: Groupby Sederhana

```
# Membuat data pengukuran tubuh dan mengelompokkannya berdasarkan usia (age)
body_measurement_df = pd.DataFrame.from_records((
    (2, 83.82, 8.4), (4, 99.31, 16.97), (3, 96.52, 14.41),
    (6, 114.3, 20.14), (4, 101.6, 16.91), (2, 86.36, 12.64),
    (3, 92.71, 14.23), (2, 85.09, 11.11), (2, 85.85, 14.18),
    (5, 106.68, 20.01), (4, 99.06, 13.17), (5, 109.22, 15.36),
    (4, 100.84, 14.78), (6, 115.06, 20.06), (2, 84.07, 10.02),
    (7, 121.67, 28.4), (3, 94.49, 14.05), (6, 116.59, 17.55),
    (7, 121.92, 22.96),
), columns=("age", "height_cm", "weight_kg"))

display(body_measurement_df.groupby(by="age").mean())
```

Code JupyterLab Python 3 (ipykernel)

[9]:

```
# Membuat data pengukuran tubuh dan mengelompokkannya berdasarkan usia (age)
body_measurement_df = pd.DataFrame.from_records((
    (2, 83.82, 8.4), (4, 99.31, 16.97), (3, 96.52, 14.41),
    (6, 114.3, 20.14), (4, 101.6, 16.91), (2, 86.36, 12.64),
    (3, 92.71, 14.23), (2, 85.09, 11.11), (2, 85.85, 14.18),
    (5, 106.68, 20.01), (4, 99.06, 13.17), (5, 109.22, 15.36),
    (4, 100.84, 14.78), (6, 115.06, 20.06), (2, 84.07, 10.02),
    (7, 121.67, 28.4), (3, 94.49, 14.05), (6, 116.59, 17.55),
    (7, 121.92, 22.96),
), columns=("age", "height_cm", "weight_kg"))

display(body_measurement_df.groupby(by="age").mean())
```

age	height_cm	weight_kg
2	85.038000	11.2700
3	94.573333	14.2300
4	100.202500	15.4575
5	107.950000	17.6850
6	115.316667	19.2500
7	121.795000	25.6800

Cell 6: Groupby dengan Agregasi (Agg)

```
# Menggunakan method agg() untuk menghitung beberapa parameter sekaligus
display(body_measurement_df.groupby(by='age').agg({
    'height_cm': 'mean',
    'weight_kg': ['mean', 'max', 'min'],
}))
```

Code JupyterLab Python 3 (ipykernel)

[11]:

```
# Menggunakan method agg() untuk menghitung beberapa parameter sekaligus
display(body_measurement_df.groupby(by='age').agg({
    'height_cm': 'mean',
    'weight_kg': ['mean', 'max', 'min'],
}))
```

	height_cm	weight_kg		
	mean	mean	max	min
age				
2	85.038000	11.2700	14.18	8.40
3	94.573333	14.2300	14.41	14.05
4	100.202500	15.4575	16.97	13.17
5	107.950000	17.6850	20.01	15.36
6	115.316667	19.2500	20.14	17.55
7	121.795000	25.6800	28.40	22.96

Bagian 3: Latihan Exploratory Data Analysis (Dicoding Collection)

Agar kode di bab ini dapat dieksekusi dengan lancar di Jupyter Notebook tanpa pesan *error*, murid perlu membuat *dummy data* terlebih dahulu, sama seperti prinsip yang dicontohkan pada langkah ke-2 pembuatan DataFrame data_tugas.

Cell 7: Persiapan Dataset Dummy (Wajib Dijalankan Pertama)

```
# ----------------------------------------------------------------------------
# KODE PERSIAPAN: Membuat Dummy Data agar script EDA dari buku dapat berjalan lancar
# ----------------------------------------------------------------------------
import pandas as pd
import numpy as np


# 1. Dummy customers_df
customers_df = pd.DataFrame({
   "customer_id": [1, 2, 3, 4, 5, 736, 231, 666, 545, 295],
   "customer_name": ["Andi", "Budi", "Citra", "Dewi", "Eka", "Ran 736", "Ian 231", "Fina 666", "Gus 545", "Prefer 295"],
```

```
    "gender": ["Male", "Male", "Female", "Female", "Prefer not to say", "Female", "Male", "Prefer not to say", "Male", "Prefer not to say"],
    "age": [25, 45, 34, 21, 60, 59, 30, 40, 22, 70],
    "city": ["New Ava", "East Aidan", "East Sophia", "Jordanside", "West Jackfort", "Tylnburg", "Port Hannahburgh", "Corkeryshire", "North Marcusland", "Port William"],
    "state": ["South Australia", "Queensland", "New South Wales", "Victoria", "Tasmania", "Western Australia", "Australian Capital Territory", "Australian Capital Territory", "Northern Territory", "Victoria"],
    "home_address": ["Alamat 1", "Alamat 2", "Alamat 3", "Alamat 4", "Alamat 5", "M77 Glover", "Port Hannah", "Corner Run", "Schweder Way", "Barton Mall"],
    "zip_code": [1001, 1002, 1003, 1004, 1005, 3600, 4527, 2022, 4054, 7499]
})

# 2. Dummy orders_df
orders_df = pd.DataFrame({
    "order_id": [712, 716, 855, 35, 645],
    "customer_id": [736, 1, 231, 2, 666],
    "payment": [26.0, 25.0, 9.0, 13.0, 20.0],
    "order_date": pd.to_datetime(["2021-07-14", "2021-03-11", "2021-01-13", "2021-08-22", "2021-05-08"]),
    "delivery_date": pd.to_datetime(["2021-08-09", "2021-04-05", "2021-01-22", "2021-09-04", "2021-05-28"])
})

# 3. Dummy product_df
product_df = pd.DataFrame({
    "product_id": [704, 671, 693, 1225, 1226, 599],
    "product_type": ["Jacket", "Jacket", "Jacket", "Trousers", "Trousers", "Jacket"],
    "product_name": ["Parka", "Parka", "Parka", "Pleated", "Pleated", "Bomber"],
    "size": ["L", "XS", "M", "XL", "XS", "L"],
    "colour": ["violet", "red", "indigo", "indigo", "violet", "violet"],
    "price": [110, 119, 119, 90, 90, 90],
    "quantity": [63, 65, 96, 45, 60, 60],
    "description": ["Deskripsi 1", "Deskripsi 2", "Deskripsi 3", "Deskripsi 4", "Deskripsi 5", "Deskripsi 6"]
})
```

 # 4. Dummy sales_df

sales_df = pd.DataFrame({

"sales_id": [1, 2, 3, 4, 5],

"order_id": [712, 716, 855, 35, 645],

"product_id": [704, 1225, 693, 1226, 599],

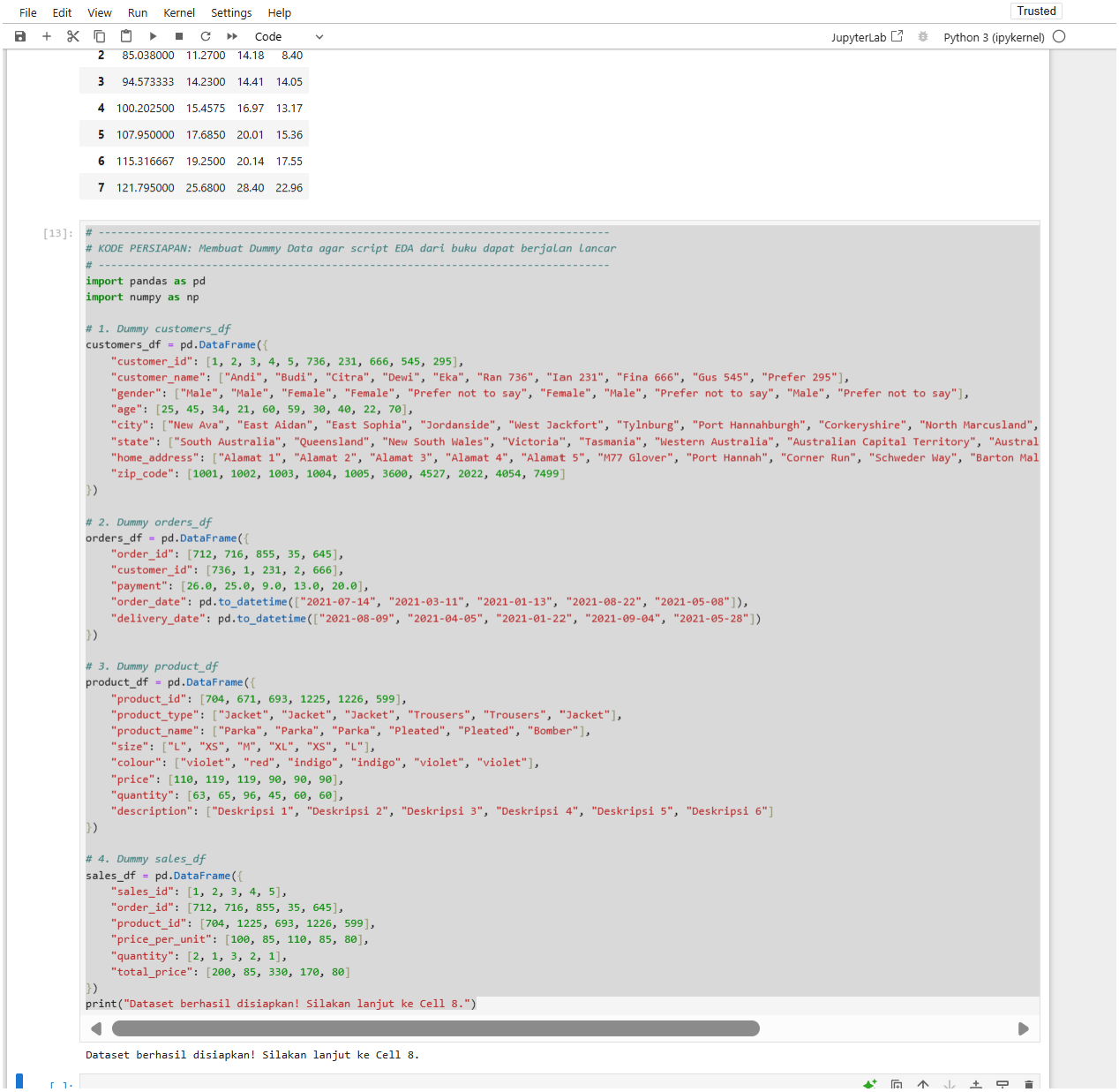
"price_per_unit": [100, 85, 110, 85, 80],

"quantity": [2, 1, 3, 2, 1],

"total_price": [200, 85, 330, 170, 80]

})

print("Dataset berhasil disiapkan! Silakan lanjut ke Cell 8.")



Cell 8: Eksplorasi customers_df

```
# Rangkuman parameter statistik dari data customers_df
display(customers_df.describe(include="all"))


# Demografi pelanggan berdasarkan gender
display(customers_df.groupby(by="gender").agg({
   "customer_id": "nunique",
   "age": ["max", "min", "mean", "std"]
}))
```

File Edit View Run Kernel Settings Help

Code JupyterLab Pyth

[15]:

```
# Rangkuman parameter statistik dari data customers_df
display(customers_df.describe(include="all"))

# Demografi pelanggan berdasarkan gender
display(customers_df.groupby(by="gender").agg({
    "customer_id": "nunique",
    "age": ["max", "min", "mean", "std"]
}))
```

	customer_id	customer_name	gender	age	city	state	home_address	zip_code
count	10.000000	10	10	10.000000	10	10	10	10.000000
unique	NaN	10	3	NaN	10	8	10	NaN
top	NaN	Andi	Male	NaN	New Ava	Victoria	Alamat 1	NaN
freq	NaN	1	4	NaN	1	2	1	NaN
mean	248.800000	NaN	NaN	40.600000	NaN	NaN	NaN	2671.700000
std	298.831725	NaN	NaN	17.411363	NaN	NaN	NaN	2207.622555
min	1.000000	NaN	NaN	21.000000	NaN	NaN	NaN	1001.000000
25%	3.250000	NaN	NaN	26.250000	NaN	NaN	NaN	1003.250000
50%	118.000000	NaN	NaN	37.000000	NaN	NaN	NaN	1513.500000
75%	482.500000	NaN	NaN	55.500000	NaN	NaN	NaN	3940.500000
max	736.000000	NaN	NaN	70.000000	NaN	NaN	NaN	7499.000000

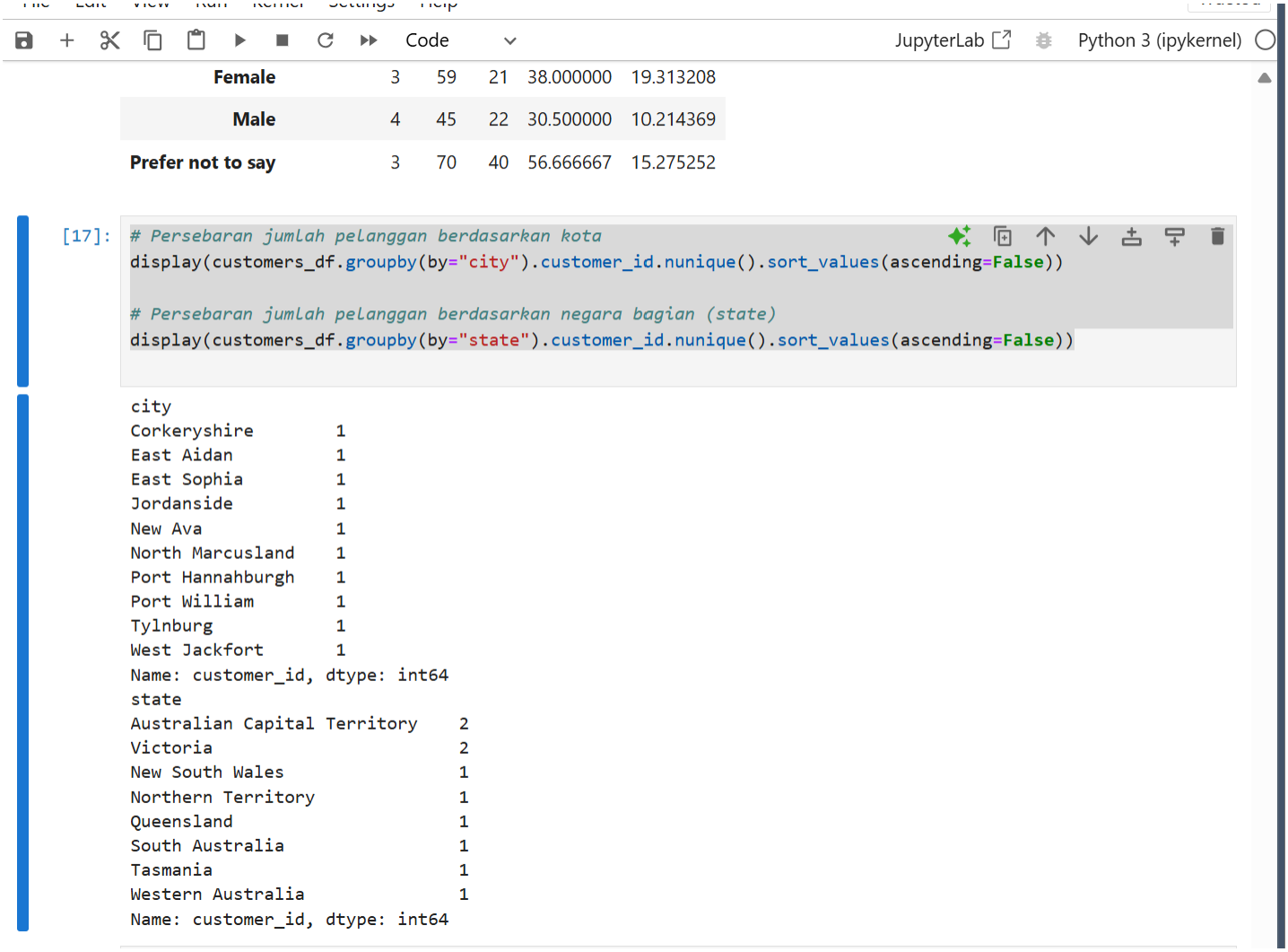
	customer_id	age			
	nunique	max	min	mean	std
gender					
Female	3	59	21	38.000000	19.313208
Male	4	45	22	30.500000	10.214369
Prefer not to say	3	70	40	56.666667	15.275252

Cell 9: Persebaran Pelanggan (City & State)

```
# Persebaran jumlah pelanggan berdasarkan kota
```

```
display(customers_df.groupby(by="city").customer_id.nunique().sort_values(ascending=False))
```

```
# Persebaran jumlah pelanggan berdasarkan negara bagian (state)
display(customers_df.groupby(by="state").customer_id.nunique().sort_values(ascending=False))
```



Female	3	59	21	38.000000	19.313208
Male	4	45	22	30.500000	10.214369
Prefer not to say	3	70	40	56.666667	15.275252

[17]:
```
# Persebaran jumlah pelanggan berdasarkan kota
display(customers_df.groupby(by="city").customer_id.nunique().sort_values(ascending=False))

# Persebaran jumlah pelanggan berdasarkan negara bagian (state)
display(customers_df.groupby(by="state").customer_id.nunique().sort_values(ascending=False))
```

```
city
Corkeryshire        1
East Aidan          1
East Sophia         1
Jordanside          1
New Ava             1
North Marcusland    1
Port Hannahburgh    1
Port William        1
Tylnburg            1
West Jackfort       1
Name: customer_id, dtype: int64
state
Australian Capital Territory    2
Victoria                        2
New South Wales                 1
Northern Territory              1
Queensland                      1
South Australia                 1
Tasmania                        1
Western Australia               1
Name: customer_id, dtype: int64
```

Cell 10: Eksplorasi orders_df (Menghitung Waktu Pengiriman)

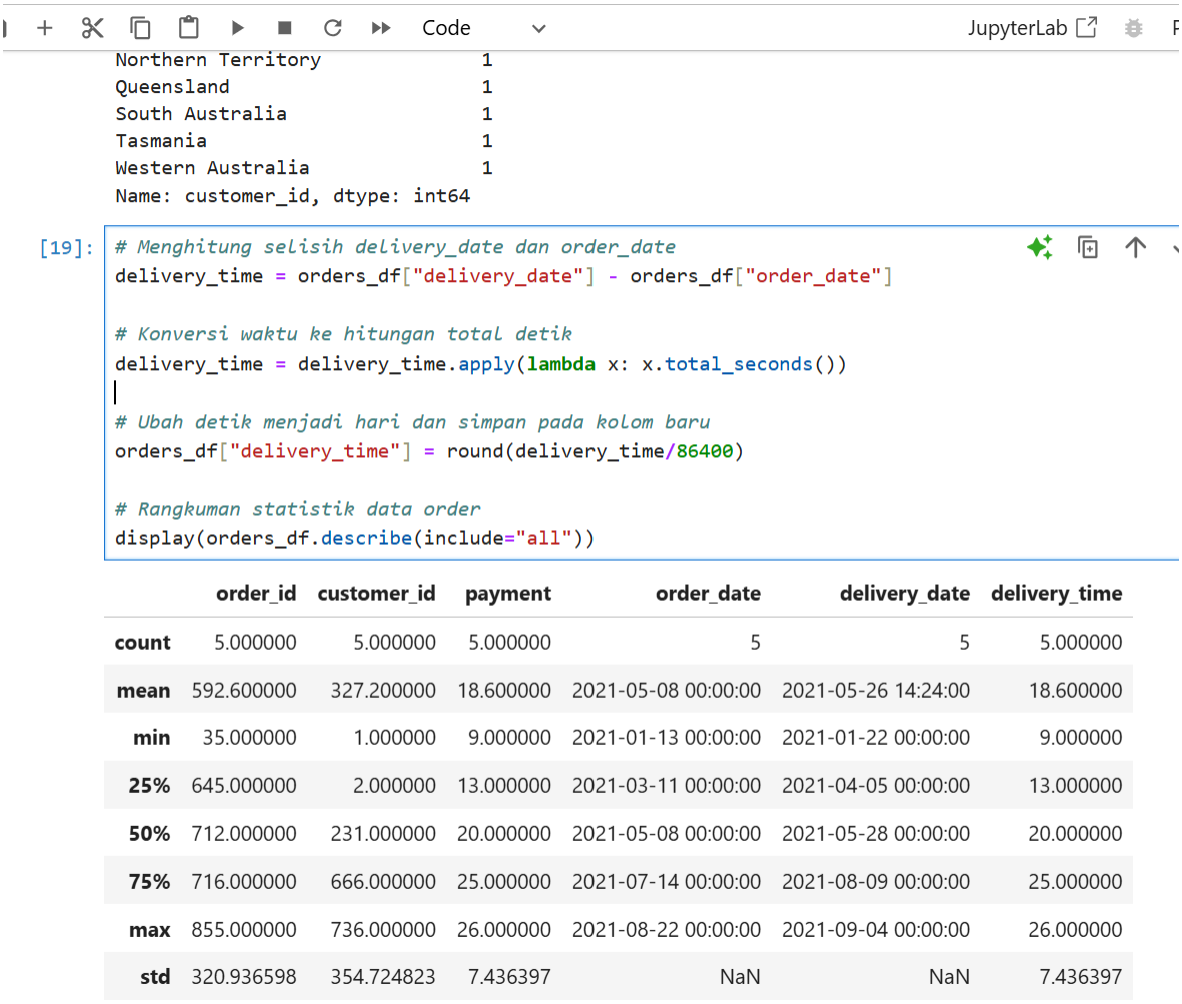
```
# Menghitung selisih delivery_date dan order_date
delivery_time = orders_df["delivery_date"] - orders_df["order_date"]
```

```
# Konversi waktu ke hitungan total detik
delivery_time = delivery_time.apply(lambda x: x.total_seconds())
```

```
# Ubah detik menjadi hari dan simpan pada kolom baru
orders_df["delivery_time"] = round(delivery_time/86400)
```

 # Rangkuman statistik data order

display(orders_df.describe(include="all"))



```
Northern Territory              1
Queensland                      1
South Australia                 1
Tasmania                        1
Western Australia               1
Name: customer_id, dtype: int64
```

[19]:

```
# Menghitung selisih delivery_date dan order_date
delivery_time = orders_df["delivery_date"] - orders_df["order_date"]

# Konversi waktu ke hitungan total detik
delivery_time = delivery_time.apply(lambda x: x.total_seconds())

# Ubah detik menjadi hari dan simpan pada kolom baru
orders_df["delivery_time"] = round(delivery_time/86400)

# Rangkuman statistik data order
display(orders_df.describe(include="all"))
```

	order_id	customer_id	payment	order_date	delivery_date	delivery_time
count	5.000000	5.000000	5.000000	5	5	5.000000
mean	592.600000	327.200000	18.600000	2021-05-08 00:00:00	2021-05-26 14:24:00	18.600000
min	35.000000	1.000000	9.000000	2021-01-13 00:00:00	2021-01-22 00:00:00	9.000000
25%	645.000000	2.000000	13.000000	2021-03-11 00:00:00	2021-04-05 00:00:00	13.000000
50%	712.000000	231.000000	20.000000	2021-05-08 00:00:00	2021-05-28 00:00:00	20.000000
75%	716.000000	666.000000	25.000000	2021-07-14 00:00:00	2021-08-09 00:00:00	25.000000
max	855.000000	736.000000	26.000000	2021-08-22 00:00:00	2021-09-04 00:00:00	26.000000
std	320.936598	354.724823	7.436397	NaN	NaN	7.436397

Cell 11: Menggabungkan orders_df & customers_df (Status Aktif)

 # Mengecek pelanggan yang sudah pernah order ("Active") dan yang belum ("Non Active")

customer_id_in_orders_df = orders_df.customer_id.tolist()

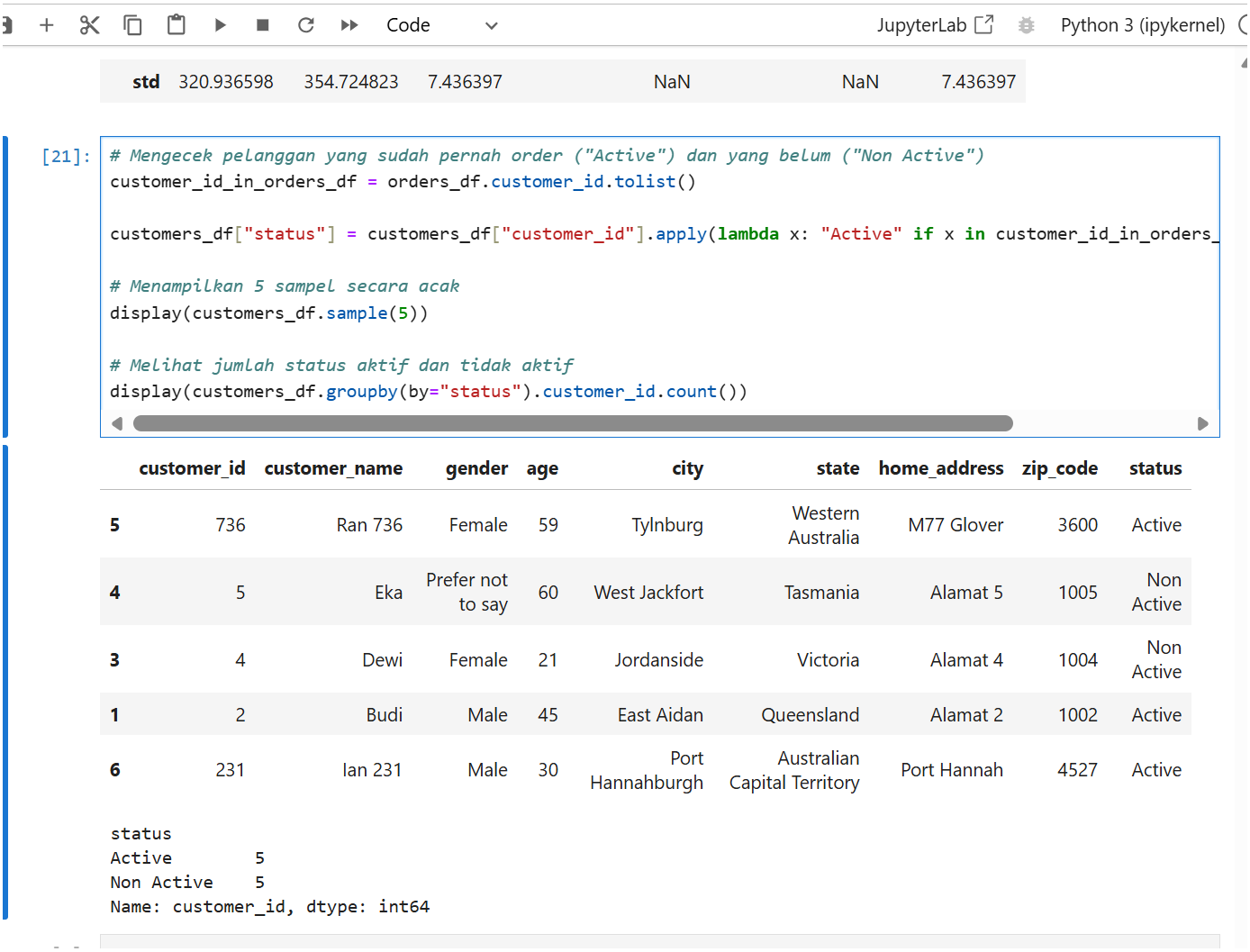
customers_df["status"] = customers_df["customer_id"].apply(lambda x: "Active" if x in customer_id_in_orders_df else "Non Active")

 # Menampilkan 5 sampel secara acak

display(customers_df.sample(5))

 # Melihat jumlah status aktif dan tidak aktif

display(customers_df.groupby(by="status").customer_id.count())



std	320.936598	354.724823	7.436397	NaN	NaN	7.436397

[21]:
```
# Mengecek pelanggan yang sudah pernah order ("Active") dan yang belum ("Non Active")
customer_id_in_orders_df = orders_df.customer_id.tolist()

customers_df["status"] = customers_df["customer_id"].apply(lambda x: "Active" if x in customer_id_in_orders_

# Menampilkan 5 sampel secara acak
display(customers_df.sample(5))

# Melihat jumlah status aktif dan tidak aktif
display(customers_df.groupby(by="status").customer_id.count())
```

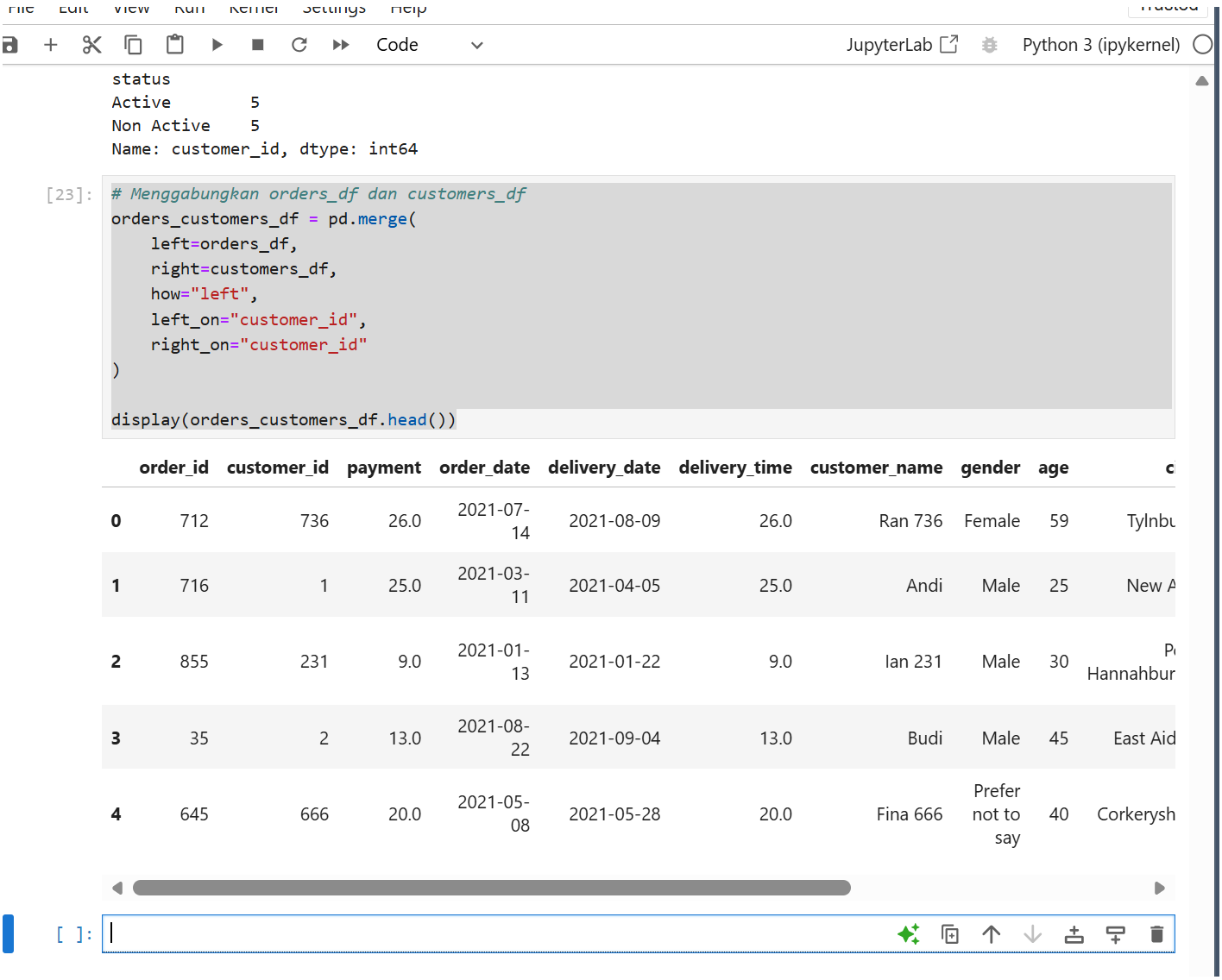
	customer_id	customer_name	gender	age	city	state	home_address	zip_code	status
5	736	Ran 736	Female	59	Tylnburg	Western Australia	M77 Glover	3600	Active
4	5	Eka	Prefer not to say	60	West Jackfort	Tasmania	Alamat 5	1005	Non Active
3	4	Dewi	Female	21	Jordanside	Victoria	Alamat 4	1004	Non Active
1	2	Budi	Male	45	East Aidan	Queensland	Alamat 2	1002	Active
6	231	Ian 231	Male	30	Port Hannahburgh	Australian Capital Territory	Port Hannah	4527	Active

```
status
Active        5
Non Active    5
Name: customer_id, dtype: int64
```

Cell 12: Merge (Join) Tabel Orders dan Customers

```
# Menggabungkan orders_df dan customers_df
orders_customers_df = pd.merge(
    left=orders_df,
    right=customers_df,
    how="left",
    left_on="customer_id",
    right_on="customer_id"
)

display(orders_customers_df.head())
```



```
status
Active        5
Non Active    5
Name: customer_id, dtype: int64
```

[23]:

```
# Menggabungkan orders_df dan customers_df
orders_customers_df = pd.merge(
    left=orders_df,
    right=customers_df,
    how="left",
    left_on="customer_id",
    right_on="customer_id"
)

display(orders_customers_df.head())
```

	order_id	customer_id	payment	order_date	delivery_date	delivery_time	customer_name	gender	age	c
0	712	736	26.0	2021-07-14	2021-08-09	26.0	Ran 736	Female	59	Tylnbu
1	716	1	25.0	2021-03-11	2021-04-05	25.0	Andi	Male	25	New A
2	855	231	9.0	2021-01-13	2021-01-22	9.0	Ian 231	Male	30	P Hannahbur
3	35	2	13.0	2021-08-22	2021-09-04	13.0	Budi	Male	45	East Aid
4	645	666	20.0	2021-05-08	2021-05-28	20.0	Fina 666	Prefer not to say	40	Corkerysh

[ ]:

Cell 13: Eksplorasi Jumlah Order (City, State, Gender)

 # Jumlah order berdasarkan kota

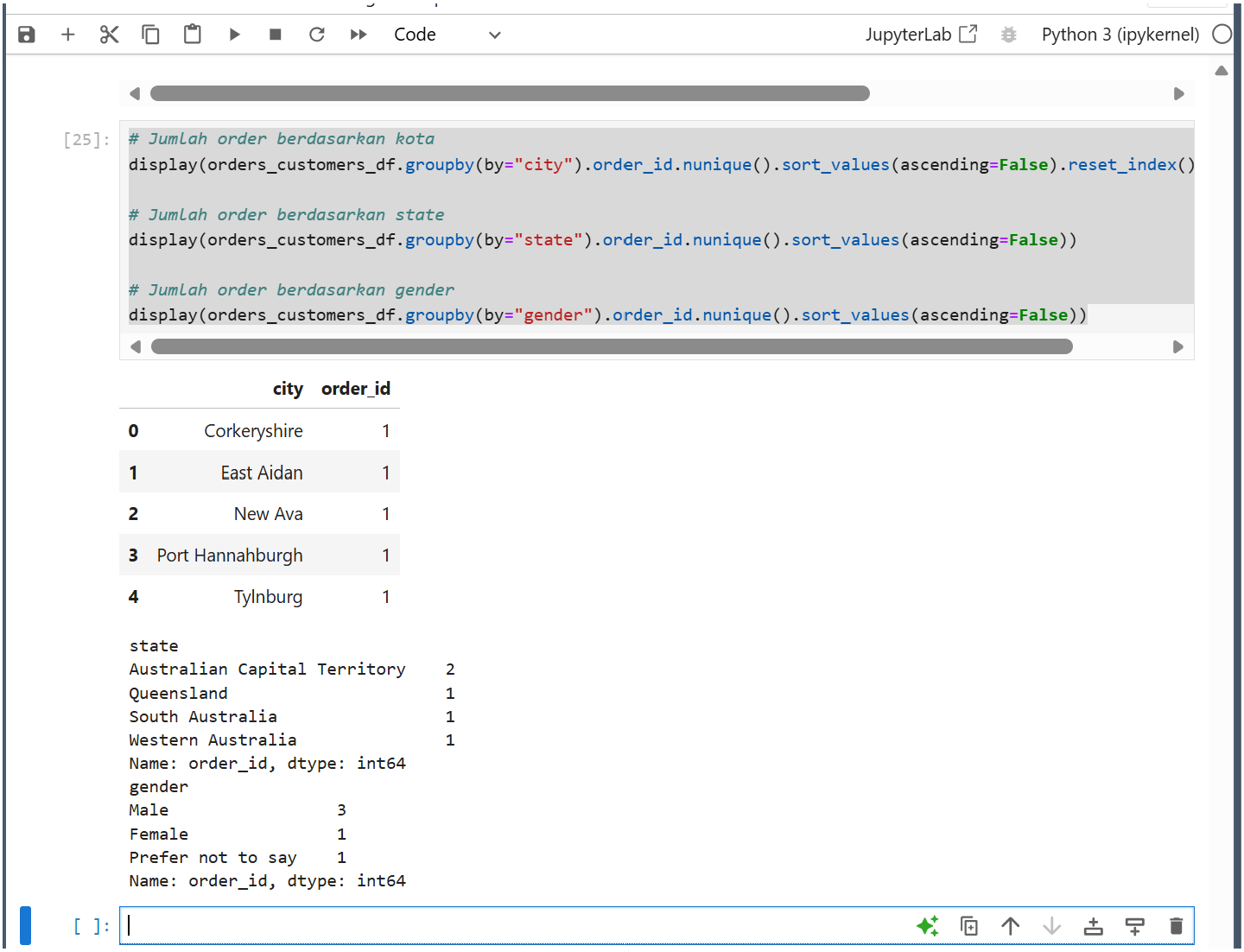
display(orders_customers_df.groupby(by="city").order_id.nunique().sort_values(ascending=False).reset_index().head(10))

 # Jumlah order berdasarkan state

display(orders_customers_df.groupby(by="state").order_id.nunique().sort_values(ascending=False))

 # Jumlah order berdasarkan gender

display(orders_customers_df.groupby(by="gender").order_id.nunique().sort_values(ascending=False))



Cell 14: Eksplorasi Berdasarkan Kelompok Usia

```
# Membuat kolom kelompok usia (Youth, Adults, Seniors)

orders_customers_df["age_group"] = orders_customers_df.age.apply(lambda x: "Youth" if x <= 24 else ("Seniors" if x > 64 else "Adults"))


# Jumlah order berdasarkan age_group

display(orders_customers_df.groupby(by="age_group").order_id.nunique().sort_values(ascending=False))
```

[27]:

```
# Membuat kolom kelompok usia (Youth, Adults, Seniors)
orders_customers_df["age_group"] = orders_customers_df.age.apply(lambda x: "Youth" if x <= 2

# Jumlah order berdasarkan age_group
display(orders_customers_df.groupby(by="age_group").order_id.nunique().sort_values(ascending
```

```
age_group
Adults    5
Name: order_id, dtype: int64
```

[ ]:

Cell 15: Eksplorasi Data product_df dan sales_df

```
# Rangkuman statistik product_df
display(product_df.describe(include="all"))

# Rangkuman statistik sales_df
display(sales_df.describe(include="all"))

# Mencari produk termahal
display(product_df.sort_values(by="price", ascending=False))
```

Code

[29]:
```
# Rangkuman statistik product_df
display(product_df.describe(include="all"))

# Rangkuman statistik sales_df
display(sales_df.describe(include="all"))

# Mencari produk termahal
display(product_df.sort_values(by="price", ascending=False))
```

	product_id	product_type	product_name	size	colour	price	quantity	description
count	6.000000	6	6	6	6	6.000000	6.000000	6
unique	NaN	2	3	4	3	NaN	NaN	6
top	NaN	Jacket	Parka	L	violet	NaN	NaN	Deskripsi 1
freq	NaN	4	3	2	3	NaN	NaN	1
mean	853.000000	NaN	NaN	NaN	NaN	103.000000	64.833333	NaN
std	290.844976	NaN	NaN	NaN	NaN	14.615061	16.821613	NaN
min	599.000000	NaN	NaN	NaN	NaN	90.000000	45.000000	NaN
25%	676.500000	NaN	NaN	NaN	NaN	90.000000	60.000000	NaN
50%	698.500000	NaN	NaN	NaN	NaN	100.000000	61.500000	NaN
75%	1094.750000	NaN	NaN	NaN	NaN	116.750000	64.500000	NaN
max	1226.000000	NaN	NaN	NaN	NaN	119.000000	96.000000	NaN

	sales_id	order_id	product_id	price_per_unit	quantity	total_price
count	5.000000	5.000000	5.000000	5.0000	5.00000	5.000000
mean	3.000000	592.600000	889.400000	92.0000	1.80000	173.000000
std	1.581139	320.936598	309.517851	12.5499	0.83666	102.200783
min	1.000000	35.000000	599.000000	80.0000	1.00000	80.000000
25%	2.000000	645.000000	693.000000	85.0000	1.00000	85.000000
50%	3.000000	712.000000	704.000000	85.0000	2.00000	170.000000
75%	4.000000	716.000000	1225.000000	100.0000	2.00000	200.000000
max	5.000000	855.000000	1226.000000	110.0000	3.00000	330.000000

	product_id	product_type	product_name	size	colour	price	quantity	description
1	671	Jacket	Parka	XS	red	119	65	Deskripsi 2
2	693	Jacket	Parka	M	indigo	119	96	Deskripsi 3
0	704	Jacket	Parka	L	violet	110	63	Deskripsi 1
3	1225	Trousers	Pleated	XL	indigo	90	45	Deskripsi 4
4	1226	Trousers	Pleated	XS	violet	90	60	Deskripsi 5
5	599	Jacket	Bomber	L	violet	90	60	Deskripsi 6

[ ]:

Cell 16: Pengelompokkan Produk (Type & Name)

```
# Pivot berdasarkan tipe produk
display(product_df.groupby(by="product_type").agg({
    "product_id": "nunique",
    "quantity": "sum",
    "price": ["min", "max"]
}))


# Pivot berdasarkan nama produk
display(product_df.groupby(by="product_name").agg({
    "product_id": "nunique",
    "quantity": "sum",
    "price": ["min", "max"]
```

}))

[31]:
```
# Pivot berdasarkan tipe produk
display(product_df.groupby(by="product_type").agg({
    "product_id": "nunique",
    "quantity": "sum",
    "price": ["min", "max"]
}))

# Pivot berdasarkan nama produk
display(product_df.groupby(by="product_name").agg({
    "product_id": "nunique",
    "quantity": "sum",
    "price": ["min", "max"]
}))
```

	product_id	quantity	price	
	nunique	sum	min	max
product_type				
Jacket	4	284	90	119
Trousers	2	105	90	90

	product_id	quantity	price	
	nunique	sum	min	max
product_name				
Bomber	1	60	90	90
Parka	3	224	110	119
Pleated	2	105	90	90

Cell 17: Merge Product dan Sales

 # Menggabungkan data sales dan product

sales_product_df = pd.merge(

left=sales_df,

right=product_df,

how="left",

left_on="product_id",

right_on="product_id"

)

display(sales_product_df.head())

 # Penjualan berdasarkan tipe produk

display(sales_product_df.groupby(by="product_type").agg({

"sales_id": "nunique",

"quantity_x": "sum",

"total_price": "sum"

}))

 # Penjualan berdasarkan nama produk (diurutkan berdasar total harga)

display(sales_product_df.groupby(by="product_name").agg({

"quantity_x": "sum",

"total_price": "sum"

}).sort_values(by="total_price", ascending=False))

[33]:
```
# Menggabungkan data sales dan product
sales_product_df = pd.merge(
    left=sales_df,
    right=product_df,
    how="left",
    left_on="product_id",
    right_on="product_id"
)

display(sales_product_df.head())

# Penjualan berdasarkan tipe produk
display(sales_product_df.groupby(by="product_type").agg({
    "sales_id": "nunique",
    "quantity_x": "sum",
    "total_price": "sum"
}))

# Penjualan berdasarkan nama produk (diurutkan berdasar total harga)
display(sales_product_df.groupby(by="product_name").agg({
    "quantity_x": "sum",
    "total_price": "sum"
}).sort_values(by="total_price", ascending=False))
```

	sales_id	order_id	product_id	price_per_unit	quantity_x	total_price	product_type	product_name	size	colour	price	quantity_y	description
0	1	712	704	100	2	200	Jacket	Parka	L	violet	110	63	Deskripsi 1
1	2	716	1225	85	1	85	Trousers	Pleated	XL	indigo	90	45	Deskripsi 4
2	3	855	693	110	3	330	Jacket	Parka	M	indigo	119	96	Deskripsi 3
3	4	35	1226	85	2	170	Trousers	Pleated	XS	violet	90	60	Deskripsi 5
4	5	645	599	80	1	80	Jacket	Bomber	L	violet	90	60	Deskripsi 6

product_type	sales_id	quantity_x	total_price
Jacket	3	6	610
Trousers	2	3	255

product_name	quantity_x	total_price
Parka	5	530
Pleated	3	255
Bomber	1	80

[ ]:

Cell 18: Menggabungkan Seluruh Data (all_df)

```
# Merge antara sales_product_df dan orders_customers_df
all_df = pd.merge(
    left=sales_product_df,
    right=orders_customers_df,
    how="left",
    left_on="order_id",
    right_on="order_id"
)

display(all_df.head())
```

[35]:

```
# Merge antara sales_product_df dan orders_customers_df
all_df = pd.merge(
    left=sales_product_df,
    right=orders_customers_df,
    how="left",
    left_on="order_id",
    right_on="order_id"
)

display(all_df.head())
```

	sales_id	order_id	product_id	price_per_unit	quantity_x	total_price	product_type	product_name	size	colour	...	delivery_time	customer_name	gender	age
0	1	712	704	100	2	200	Jacket	Parka	L	violet	...	26.0	Ran 736	Female	59
1	2	716	1225	85	1	85	Trousers	Pleated	XL	indigo	...	25.0	Andi	Male	25
2	3	855	693	110	3	330	Jacket	Parka	M	indigo	...	9.0	Ian 231	Male	30
3	4	35	1226	85	2	170	Trousers	Pleated	XS	violet	...	13.0	Budi	Male	45
4	5	645	599	80	1	80	Jacket	Bomber	L	violet	...	20.0	Fina 666	Prefer not to say	40

5 rows × 27 columns

Cell 19: Eksplorasi Pola Pembelian (State, Gender, Age Group)

```
# Preferensi pembelian berdasarkan state dan tipe produk
display(all_df.groupby(by=["state", "product_type"]).agg({
    "quantity_x": "sum",
    "total_price": "sum"
}))

# Preferensi berdasarkan gender dan tipe produk
display(all_df.groupby(by=["gender", "product_type"]).agg({
    "quantity_x": "sum",
    "total_price": "sum"
```

}))

Preferensi berdasarkan usia dan tipe produk

display(all_df.groupby(by=["age_group", "product_type"]).agg({

"quantity_x": "sum",

"total_price": "sum"

}))

[37]:
```
# Preferensi pembelian berdasarkan state dan tipe produk
display(all_df.groupby(by=["state", "product_type"]).agg({
    "quantity_x": "sum",
    "total_price": "sum"
}))

# Preferensi berdasarkan gender dan tipe produk
display(all_df.groupby(by=["gender", "product_type"]).agg({
    "quantity_x": "sum",
    "total_price": "sum"
}))

# Preferensi berdasarkan usia dan tipe produk
display(all_df.groupby(by=["age_group", "product_type"]).agg({
    "quantity_x": "sum",
    "total_price": "sum"
}))
```

		quantity_x	total_price
state	product_type		
Australian Capital Territory	Jacket	4	410
Queensland	Trousers	2	170
South Australia	Trousers	1	85
Western Australia	Jacket	2	200

		quantity_x	total_price
gender	product_type		
Female	Jacket	2	200
Male	Jacket	3	330
	Trousers	3	255
Prefer not to say	Jacket	1	80

		quantity_x	total_price
age_group	product_type		
Adults	Jacket	6	610
	Trousers	3	255

[ ]: